

PARALLEL COMPUTING — CLASS 0 DIAGNOSTIC

Prerequisite & problem-solving diagnostic • 2 hours

Purpose: This session checks your current programming, algorithmic, data, and systems knowledge. You are not expected to know parallel computing yet. Work individually and show your reasoning.

Instructions: Use a language you know. Do not worry about perfect syntax unless asked. For algorithm questions, explain your thinking. If you do not know something, say so and move on.

Task 1 — Trace the Code (5 min)

Determine the final output. Show the value of x and y after each statement.

```
x = 5
```

```
y = 2
```

```
x = x + y
```

```
y = x * 2
```

```
x = y - x
```

```
print(x, y)
```

```
7,14
```

Task 2 — Find the Bug (5 min)

What does this program actually calculate? What was it probably intended to calculate? Fix it.

I think it should print total count of this array

```
numbers = [10, 20, 30, 40, 50]
```

```
total = 0
```

```
for i in range(len(numbers)):
    total = numbers[i]+total
```

```
print(total)
```

Task 3 — Nested Loops (5 min)

For the program below: (a) how many times does print() execute? (b) what if both loops run from 0 to 999? (c) what if the outer loop runs 10 times and the inner loop runs 1,000,000 times?

a)100 b) 1,000,000 c) 10,000,000

```
for i in range(10):
    for j in range(10):
        print(i, j)
```

Part B — Algorithms & Complexity

Task 4 — Which Is Faster? (5 min)

Algorithm A scans an array once. Algorithm B compares every element with every element. For an array of size n , estimate the amount of work for each. Which grows faster? What happens when $n = 1,000,000$?

algorithm A scans work will be $= n$, algorithm B work $= n^2$ so A = 1,000,000 operations, B= 1,000,000,000,000 operations

Task 5 — Complexity Ranking (7 min)

Rank these from fastest-growing to slowest-growing as n increases:

n^2 $\log_2 n$ n $n \log_2 n$ 1 2^n

Which become particularly problematic when n becomes much larger?

2^n

n^2

$n \log_2 n$

n

$\log_2 n$

1

problematic is

n^2

2^n

Task 6 — Searching (7 min)

Given [3, 8, 12, 17, 24, 31, 45, 51, 63], find 45 using two approaches: (1) if you do not know the list is sorted; (2) if you know it is sorted. Explain why the second can be faster.

Know that it is sorted we can use binary search with starting in the middle. So middle is 24, 45 is larger in the right side the we can found. binary search can be much faster for very large sorted datasets

Part C — Data & Problem Solving

Task 7 — Large Dataset (8 min)

You have 500 million integers and need to find the largest number. Design an algorithm. How many times does each number need to be examined? Do you need to store all 500 million numbers? Could you process the data in chunks?

Task 8 — Counting (5 min)

Given A B A C B A D C A B, design an algorithm producing $A \rightarrow 4$, $B \rightarrow 3$, $C \rightarrow 2$, $D \rightarrow 1$. What data structure would you use?

counts = {}

for letter in data:

 if letter in counts:

 counts[letter] += 1

 else:

 counts[letter] = 1

dictionary data structure

Task 9 — Matrix Operations (5 min)

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. Calculate $A+B$ and $A \times B$. How many individual multiplication operations are required for two 2×2 matrices?

8

Part D — Systems Intuition

Task 10 — CPU vs Memory (6 min)

A program scans a huge array, but the CPU is extremely fast and the program is still slow. Give three possible reasons.

1 Memory bandwidth – CPU is waiting for data from RAM.

2 Cache misses – required data isn't available in fast CPU cache.

3 Huge amount of data – even a fast CPU needs time to process billions of values.

```
for x in huge_array:
    total += x
```

Task 11 — What Happens When You Run a Program? (7 min)

Describe what happens between typing `./program` and seeing the program's output. Use ordinary language. Include anything you think is relevant.

1. You type:

`./program`

2. The operating system receives the request.

3. It finds the executable program.

4. The OS creates a process for the program.

5. Program instructions and required data are loaded into memory.
6. The CPU starts executing instructions.
7. The program may request resources such as memory, files, network access, etc.
8. The CPU executes the program's instructions.
9. The program produces output.
10. The output is sent to the terminal/display.
11. The program eventually finishes and the OS releases its resources.

Task 12 — Four Programs (7 min)

Four programs run simultaneously: A uses lots of CPU; B uses lots of memory; C reads a large file; D mostly waits for user input. Identify the likely bottleneck for each. Can all four run at the same time? What determines CPU time? What changes if there is one CPU core?

Yes, all four can run at the same time.

The operating system manages them and decides which process gets CPU time.

With multiple CPU cores, different programs/threads can actually execute simultaneously on different cores.

With one CPU core, only one thread executes at a given instant, but the OS rapidly switches between programs, creating the appearance that they run simultaneously.

Part E — Reasoning Challenges

Task 13 — The Slow Program (6 min)

A program processes 1 billion records in 10 hours. A student says: “If we buy a computer that is twice as fast, the program will take 5 hours.” Do you agree? Explain.

I don't agree with this because programs may also be limited by memory speed, database networks and so on.

Task 14 — Make It Faster (6 min)

A program takes 60 seconds. You may change anything except the answer it produces. List as many ways as possible to reduce execution time.

Reduce unnecessary loops, improve algorithm, using better data structure and so on

Task 15 — Bottleneck Identification (6 min)

Execution times are: A=2s, B=2s, C=50s, D=3s, E=2s. Total=59s. If A, B, D and E become 10× faster, how much faster does the entire program become? Where should optimization effort go?

$59 / 50.9 \approx 1.16\times$ Optimization effort should go to C, because C is the bottleneck.

Final Challenge — Design the Algorithm

Task 16 — 10 Million Student Marks (10 min)

You receive 10 million student marks, each between 0 and 100. You must produce: minimum, maximum, average, median, number scoring ≥ 90 , and number scoring < 40 . Design an algorithm; identify data structures; estimate how many times the data must be examined; estimate memory requirements; identify the likely bottleneck; and explain how you would make it faster without changing the answer.

minimum = 101

maximum = -1

total = 0

count_90 = 0

count_40 = 0

for mark in marks:

 minimum = min(minimum, mark)

```
maximum = max(maximum, mark)
total += mark
```

```
if mark >= 90:
    count_90 += 1
```

```
if mark < 40:
    count_40 += 1
```

```
average = total / len(marks)
```

I would go through all 10 million marks once.

- Find the **minimum** and **maximum**.
- Add all marks to calculate the **average**.
- Count marks ≥ 90 and < 40 .
- For the **median**, I would use a frequency array because marks are only from 0 to 100.
- I need to examine each mark **once**, so about **10 million times**.
- I don't need to store all marks; I can process them in **chunks**.
- The main bottleneck is processing the large amount of data.
- To make it faster, I can use **chunks and parallel processing**.

Data structure: an array of 101 counters (0-100).

STOP HERE. Check that you attempted every task. You are not expected to know parallel computing yet.